Program 6

Aim:- Implementation of class diagram for LMS.

Class Diagrams:-

Terms & conditions-

A class is a descriptions of a set of objects that share the same attributes, operations, relationships & semantics. Graphically contains the following things which is rendered as a ractangle.

Classes,
Interfaces,
Collaborations
Dependancy, Generalization, Association & Relationships.

Class diagrams may also contains packages or subsystems both of which are used to group elements of your model into layer chunks.

Class diagrams are used in one of 3 ways.

Modeling in the Distribution of Responsibilities in a system.
Modeling the distribution of a System.
Modeling Non-software Things.

Common modeling techniques:-

You'll use classes most commonly to modely abstractions that are drawn from the problem, you are trying to solve or from the technology you are using to implement a solution to that problem. Each of these abstractions is a part of the vocabulary of your system, meaning that, together, they represent the things that are important to users & to implementers.

Modeling Non-Software Things:-

To model the vocabulary of a System.

- Identify those things that users or implementers
- use to describe the problem or solution.
- Use CRC cards & use case-based analysis to help find these abstractions.
- For each abstraction, identify a set of responsibility. Make sure that each class is crisply defined and that there is a good balance of responsibilities among all your classes.
- Provide the attributes and operations that are needed to carry out these responsibilities for each class.

Modeling the Distribution of Responsibilities in a System:-

To model the distribution of responsibility in a system.

- Identify a set of classes that work together closely to carry out some behavior.
- Identify a set of responsibilities for each of these classes. Look at this set of classes as a whole, split classes that have too many responsibilities into larger

ones & reallocate responsibilities so that each abstraction reasonably stands on its own.
- Consider the ways in which those classes collaborate with one another, and redistribute their responsibilities accordingly so that no class within a collaboration does too much or too little.

Modeling Non-Software Things :-

To model non-software things -
- Model the thing you are abstracting as a class.
- If you want to distinguish these things from the UML's defined building blocks, create a new building block by using stereotypes to specify these new semantics & to give a distinctive visual cue.

If the thing you are modeling is some kind of hardware that itself contains software, consider modeling it as a kind of node, as well, so that you can further expand on its structure.

Viva Questions:-

Q1 Explain class & objects?

Ans Class: A class describes the contents of the objects that belong to it. It describes an aggregate of data fields (called instance variable), and defines the operations (called methods)

Object: an object is an element (or instance) of a ~~glass~~ class, objects have the behaviours of their class.

Q2 What do you understand by class diagram?
Ans This diagram explores details design of the system. The class diagram is designed ~~to~~ using Use case diagram. We can identify all Nouns in use cases as classes and verbs as method of the classes.

Q3 How many types of UML Diagram?
Ans Various types of UML Diagram are listen below:-

1. Use case Diagram
2. Class Diagram
3. Object diagram
4. State Diagram
5. Sequence Diagram
6. Collaboration Diagram
7. Activity Diagram
8. Component Diagram
9. Deployment diagram

Q4 Why UML diagram is important?
Ans An object contains both data & methods that control the data. UML is powerful enough to represent all the concepts that exist in object-oriented analysis and design. UML diagrams are representation of object-oriented concepts only. Thus, before learning UML, it becomes important to understand OO concept in details.

Q5 Why do we use class diagram?
Ans Class diagram describes the attributes & operations of a class & also the constraints imposed on the system. The class diagrams are widely used in the modeling of object-oriented systems because they are the only UML diagrams, which can be mapped directly with object oriented-language.